Case 02 · Spec: Maturity Wall Simulator

Chapter 2 (Specification) of the Magshimim project-book template: features, technologies, flow diagram and database structure. The document is written so that another developer could build the system from it. Each feature is described at the level of "code in words": what it does, how it works step by step, which functions and data structures, which industry tools complement it, and what its inputs and outputs are.

System overview

What the system does, in one sentence

Maturity Wall Simulator loads a country's government bonds from an official source, bond by bond, and computes forward how each one will be replaced, at what rate, and what that will do to interest payments and debt, under any yield scenario the user chooses.

What changed since the POC and the threat model

- The POC proved that a bond-by-bond calculation gives a different answer from simple approximations. The question left open: does it also differ from a professional IMF-style approximation? Before the spec we checked: an IMF-style approximation that uses the real maturity profile was still off by up to 0.53 points of GDP in the "yields stay" scenario, and by up to 0.8 in the "up" scenario. The main reason: short bonds issued after today mature again and again, and the approximation does not see it. Feature 7 turns this check into part of the system.
- **The threat model found that the main threat is credibility:** a quoted number must prove where it came from. Features 2, 3 and 9 are built around that.

Feature list

- Feature 1, Source Adapters
- Feature 2, Normalization & Reconciliation
- Feature 3, Versioned Store & Provenance
- Feature 4, Curve & Scenario Builder
- Feature 5, Rollover Engine
- Feature 6, Debt Dynamics Engine (r versus g)
- Feature 7, Method Comparison
- Feature 8, Maturity Wall View
- Feature 9, Reproducible Permalinks & Signed Results
- Feature 10, Scheduled Refresh & Change Detection

Feature 1: Source Adapters

What does the feature do?

Downloads each country's list of marketable bonds from its debt office, together with the current yield curve, and converts every source into a uniform intermediate format. Every country publishes differently: America through a public API, France and Italy in monthly Excel files. This feature hides those differences from the rest of the system.

How does it work?

A uniform interface for every source

Each source is implemented as a class inheriting from SourceAdapter, an abstract class with three methods: fetch(), which downloads the raw file and stores it as is; parse(), which converts it into a list of RawBond; and declared_total(), which returns the total marketable debt the country itself reports (for the completeness check in feature 2). Classes register in an ADAPTERS dictionary by country code (ISO 3166, such as "US", "FR", "IT"), so adding a country means adding one class without touching the rest of the code.

America: the Fiscal Data API

The US Treasury publishes table 3 of the Monthly Statement of the Public Debt (Detail of Marketable Treasury Securities Outstanding) through its public API, at the endpoint v1/debt/mspd/mspd_table_3_market. Each record describes one bond series, with the fields security_class1_desc (type: Bills, Notes, Bonds, TIPS, FRN), security_class2_desc (CUSIP identifier), interest_rate_pct (coupon), yield_pct, issue_date, maturity_date, outstanding_amt (in millions of dollars) and inflation_adj_amt (the indexation add-on for TIPS).

The adapter sends a GET request with filter=record_date:eq:<YYYY-MM-DD> (the last published month-end) and page[size]=10000, and pages through the links.next field in the response until there are no more pages. Requests go through httpx with a 30-second timeout, up to 3 retries with exponential backoff (2, 4 and 8 seconds), and an identifying User-Agent header.

France and Italy: monthly Excel files

Agence France Trésor publishes a monthly file of marketable debt (long OATs, short BTFs, and inflation-linked OATi/OAT€i). The Italian Treasury publishes a similar file for BTPs and BOTs. The adapter downloads the file, stores it as is, and reads it with openpyxl. Because the sheet layout can change, the adapter does not rely on column numbers: it searches for the header row by keywords ("Échéance", "Encours", "Coupon") and builds a dynamic column map. If not all required headers are found, the adapter fails explicitly rather than guessing.

The yield curve

For Europe: the ECB publishes in its YC dataset the daily Svensson curve parameters for euro-area government bonds (series such as YC.B.U2.EUR.4F.G_N_A.SV_C_YM.BETA0 through TAU2 for the AAA curve, and G_N_C for all countries). The adapter downloads the six parameters for the latest day. For each country's specific spread over the European curve, the system uses the yields of that country's own bonds, already in the list. For America: the Treasury's daily par yield curve.

Keeping the raw source

Every raw file is stored as is, before any processing, with a SHA-256 fingerprint of its contents and the download date. This is the basis of the provenance chain in feature 3: it is always possible to go back to the original file a number was derived from.

Complementary industry technologies and infrastructure

- **httpx with tenacity:** httpx is a modern Python HTTP library supporting synchronous and asynchronous work, HTTP/2 and precise timeout control. tenacity is a dedicated retry library

with backoff policies. Contribution: an official source that is briefly unavailable does not break the whole refresh, and a retry does not flood their server.

- **Pydantic:** a data-validation library in which every structure is a typed class. Every RawBond an adapter returns is type-checked immediately (a date is a date, an amount is a positive number). Contribution: a parsing error is caught next to its source, not three stages later as a strange number on a chart.
- **Great Expectations (as inspiration):** a leading industry tool for data-quality checks (data contracts). We adopt the idea: each source has a "contract" of expectations (a plausible coupon range, maturity after issue), checked on every load.

Data structure

```
class RawBond(BaseModel):
    source_id: str          # "US_MSPD_T3", "FR_AFT_MONTHLY"
    instrument_id: str      # CUSIP / ISIN
    kind: Literal["bill", "fixed", "linker", "floater"]
    currency: str           # ISO 4217
    coupon_pct: float | None
    issue_date: date
    maturity_date: date
    outstanding: Decimal    # in currency units, not millions
    index_ratio: float | None   # linkers only

class FetchRecord:
    source_id: str
    url: str
    fetched_at: datetime
    raw_sha256: str
    raw_path: str
```

Input and output

Input: a country code and a report date. Output: a list of RawBond, the declared total, a raw yield curve, and a FetchRecord for every downloaded file.

Key libraries and functions

Operation	Library / API	Usage
API download	httpx	client.get() with params and paging
Retries	tenacity	@retry(wait=wait_exponential, stop=stop_after_attempt(3))
Read Excel	openpyxl	load_workbook(read_only=True), iter_rows()
Type validation	Pydantic	BaseModel, field_validator
Fingerprint	hashlib	sha256(raw_bytes).hexdigest()

Feature 2: Normalization & Reconciliation

What does the feature do?

Converts each country's RawBond records into a single, uniform, clean Bond object, and verifies that the list is complete and correct before anything is built on it. This is the line of defence against the "tampering during load" threat from the threat model: an adapter that read the wrong column, or a file that changed silently.

How does it work?

Normalisation

Every amount is converted from the source's unit (millions, billions) to whole currency units and stored as Decimal, not float, to avoid rounding errors when adding thousands of bonds. Dates are normalised to date. For inflation-linked bonds both the original principal and the adjusted principal (outstanding times index_ratio) are kept, because interest is computed on the adjusted one. Duplicate identifiers (the same ISIN on two rows, for instance because of a re-opening of the same series) are merged into one series with a cumulative amount.

Reconciliation against declared debt

After normalisation, the feature sums all outstanding amounts and compares them with that country's declared_total(). Tolerance: 0.5%. Above it, the load is marked REJECTED and does not enter the store. Below it, the gap is kept as reconciliation_gap and shown in the interface next to every result ("the list covers 99.7% of reported debt").

Plausibility checks (data contract)

For each bond: maturity_date after issue_date; coupon between 0 and 15 percent (empty only for bills); positive outstanding; currency matching the country. For the whole load: the number of bonds has not changed by more than 10% since the previous load; the weighted average coupon has not moved by more than 0.3 points in a month. Every violation is recorded as a ValidationIssue with a severity (WARN or BLOCK). A BLOCK violation stops the load.

Comparison with the previous load

The feature computes a diff between the new list and the previous one by instrument_id: bonds added (new issues), bonds that disappeared (matured or redeemed), and bonds whose amount changed. A bond that disappears before its maturity date is an anomaly flagged for review (possibly early redemption, possibly a reading error).

Complementary industry technologies and infrastructure

- **Pandera:** a schema library for data tables (DataFrames) that defines rules per column and for the whole table, and checks them all at once. Contribution: plausibility rules are written once as a declared schema, not scattered as ifs in the code.
- **Polars:** a columnar table library written in Rust. The diff between two loads is a join on instrument_id, and Polars does it quickly even on tens of thousands of rows. Chosen over pandas for stricter typing and performance.
- **The banking principle of reconciliation:** in every serious financial system, every process ends by reconciling against an independent source ("closing the books"). We adopt the principle literally: no result without reconciliation against what the country itself declares.

Data structure

```
@dataclass(frozen=True)
class Bond:
    instrument_id: str
    country: str
    kind: str              # bill | fixed | linker | floater
    coupon_pct: Decimal
    issue_date: date
    maturity_date: date
    face: Decimal          # outstanding, inflation-adjusted for linkers
    orig_tenor_y: int      # rounded (maturity - issue) in years

class ValidationIssue:
    rule_id: str           # "COUPON_RANGE", "COUNT_JUMP", ...
    severity: Literal["WARN", "BLOCK"]
    instrument_id: str | None
    detail: str
```

Input and output

Input: the RawBond list and declared total from feature 1, and the previous load from the store. Output: a validated Bond list, reconciliation_gap, a list of ValidationIssue, and a diff against the previous load. Or a REJECTED decision with reasons.

Key libraries and functions

Operation	Library	Usage
Exact arithmetic	decimal	Decimal, quantize()
Schemas and checks	Pandera	DataFrameSchema, Check
Comparing loads	Polars	join(how="full"), filter()

Feature 3: Versioned Store & Provenance

What does the feature do?

Stores every approved load as a snapshot that never changes, and keeps the whole chain for each snapshot: which raw file it came from, which adapter version, and which checks it passed. So every number the system produces can be traced back to the official file it came from. This answers the "repudiating the source" threat.

How does it work?

Immutable snapshots

Every approved load gets a snapshot_id, the SHA-256 fingerprint of the normalised Bond list. To make the fingerprint deterministic, the list is sorted by instrument_id and serialised to canonical JSON (sorted keys, no whitespace, numbers as Decimal strings) before hashing. The same list always gives the same identifier, and changing one bond changes it. Records are never deleted or updated: a new load is a new snapshot.

The provenance chain

For each snapshot a Provenance record is kept: the FetchRecord identifiers of the raw files (with their SHA-256), the adapter code version (git commit), the list of ValidationIssue, and reconciliation_gap. The structure is inspired by the W3C PROV standard: an entity (snapshot) was derived from an entity (raw file) by an activity (load) carried out by an agent (an adapter at a given version).

Storage

The store is DuckDB, an analytical database that runs inside the process and keeps everything in one file. Raw files are stored in a folder keyed by raw_sha256 (content-addressed storage), so an identical file downloaded twice is stored once.

Complementary industry technologies and infrastructure

- **DuckDB:** "the SQLite of analytics". Runs in-process with no server, columnar, and very fast at aggregation queries on large tables. Contribution: the entire history of every country can be queried in plain SQL and exported as one Parquet file per snapshot.
- **W3C PROV:** a standard for describing the provenance of data, used by research repositories and governments. We do not implement the whole standard, only its three-concept model (Entity, Activity, Agent). Contribution: a familiar, clear structure instead of an invention.
- **Content-addressed storage:** the principle Git is built on: a file is identified by the fingerprint of its content. A file cannot change without its identifier changing, so a "silent change" is impossible.

Data structure

```
Snapshot {
  snapshot_id: str          # sha256 of canonical bond list
  country: str
  as_of: date               # report date
  created_at: datetime
  bond_count: int
  total_face: Decimal
  reconciliation_gap_pct: float
}

Provenance {
  snapshot_id: str
  fetch_ids: list[str]      # -> FetchRecord
  adapter_version: str      # git commit sha
  issues: list[ValidationIssue]
}
```

Input and output

Input: a validated Bond list and check data from feature 2. Output: a fixed snapshot_id, and rows in the store that never change.

Key libraries and functions

Operation	Library	Usage
Analytical store	DuckDB	duckdb.connect(), execute(), Parquet export

Operation	Library	Usage
Canonical JSON	json	dumps(sort_keys=True, separators=(",", ":"))
Fingerprint	hashlib	sha256

Feature 4: Curve & Scenario Builder

What does the feature do?

The rollover engine needs to know, for every future year and every bond tenor, at what yield a new bond will be issued. This feature builds a continuous yield curve from today's data and lets the user define scenarios: how that curve moves over the next ten years.

How does it work?

The Nelson-Siegel-Svensson curve

A yield curve is a function that returns a yield for every maturity (one year, 5 years, 30 years). The standard method at central banks, including the ECB, is the Nelson-Siegel-Svensson model: six parameters (four betas and two taus) describing the curve's level, slope and curvature. For Europe the parameters come ready from the ECB. For each country's spread over the European curve, the feature fits a curve of its own to that country's bond yields (yield_pct, or a yield computed from price), using scipy.optimize.least_squares to minimise the sum of squared errors between the curve's yield and each bond's actual yield. The result: a function curve(tenor) returning a yield for any maturity.

Scenarios

A scenario is a path of changes to the curve over time, year by year. Four built-in types:

- **Parallel shift:** the whole curve moves by the same amount (for example up one point).
- **Steepener / Flattener:** long yields move more or less than short ones.
- **Path:** a custom path, such as a gradual rise of a quarter point a year.
- **Forward:** the future curve the market itself implies today (forward rates), in other words "what the market expects".

Each scenario also carries macro assumptions: nominal growth and the primary balance for each year. Defaults are taken from official forecasts (the IMF World Economic Outlook and European Commission forecasts), with a link to the source.

The issuance mix

At what tenor will a bond replacing a maturing one be issued? The debt office decides. Default: the mix of the last three years, as derived from the bonds issued in that period in the list itself. The user can change it (for example "France shifts towards shorter issuance"). The mix is a visible parameter, not a hidden assumption, as the POC required.

Complementary industry technologies and infrastructure

- **SciPy (least_squares with the Trust Region Reflective method):** solves non-linear least-squares problems with parameter bounds (a positive tau, say). Contribution: stable, fast curve fitting that does not wander off to meaningless parameters.
- **QuantLib (as an alternative and a check):** the long-established, widely used open library for pricing derivatives and bonds, used by banks and institutions. It includes curve bootstrapping and NSS models. Contribution: a cross-check. The curve we build is compared with the one QuantLib builds from the same data, and the gap must be negligible.
- **The ECB's Svensson model:** using the same method the central bank itself uses to publish its curve. Contribution: a professional reader can compare our curve with the official publication.

Data structure

```
NSSParams { beta0, beta1, beta2, beta3: float; tau1, tau2: float }

Curve {
  curve_id: str
  base: Literal["ECB_AAA", "ECB_ALL", "UST_PAR", "FITTED"]
  as_of: date
  params: NSSParams
  spread_params: NSSParams | None   # country spread over base
  fit_rmse_bp: float
}

Scenario {
  scenario_id: str           # sha256 of canonical definition
  name: str
  curve_id: str
  shocks: list[YearShock]    # per year: level_bp, slope_bp
  growth_nominal_pct: list[float]
  primary_balance_pct: list[float]
  issuance_mix: dict[int, float]   # tenor -> share, sums to 1
  source_notes: list[str]
}
```

Input and output

Input: curve parameters from the ECB or the Treasury, bond yields from the list, and the user's choices. Output: a fitted Curve, a Scenario with a fixed identifier, and a function yield_at(year, tenor).

Key libraries and functions

Operation	Library	Usage
Curve fitting	SciPy	optimize.least_squares(method="trf", bounds=...)
Cross-check	QuantLib	FittedBondDiscountCurve, SvenssonFitting
Vector evaluation	NumPy	evaluating the curve at all tenors at once

Feature 5: Rollover Engine

What does the feature do?

This is the technological core. The engine takes a snapshot and a scenario and runs the future month by month: every maturing bond is replaced by a new bond at the scenario's yield, every deficit is financed by new issuance, and every coupon payment is recorded. The output is the full path of debt and interest payments, at the level of the individual bond.

How does it work?

A monthly time step

The POC worked in annual steps. In the full system the step is monthly, because maturity walls cluster in particular months (American notes maturing on the 15th of February, May, August and November, for example), and short bills (one, three and six months) roll over several times a year. Ten years is 120 steps.

In-memory data structure

Instead of a list of objects, the portfolio is held as parallel NumPy arrays (a Structure of Arrays): face, coupon, maturity_month, issue_month, kind, tenor. So every operation over the whole portfolio is one vector operation. To find quickly what matures each month, a maturing_index dictionary maps each month to a list of indices, and is updated when a new bond is issued. This turns the maturity lookup from O(n) per step into O(1).

What happens at each step

- **Interest:** compute the month's coupons for all active bonds. A fixed bond pays coupons on its schedule (semi-annual in America, annual in Europe). A bill pays no coupon; its cost is the discount at issue, recorded as interest in the month it matures. A linker pays its coupon on the adjusted principal, and the principal itself grows with the scenario's inflation. A floater pays according to the scenario's short rate that month.
- **Maturities:** every bond in that month's index leaves the portfolio.
- **Financing need:** the sum of maturities plus the month's overall deficit (the scenario's primary deficit divided by 12, plus interest paid).
- **Issuance:** the need is split according to the issuance mix. For each tenor a new bond is created with a coupon equal to yield_at(month, tenor) from the scenario's curve, rounded to 0.125 as is customary in real auctions.
- **Recording:** for each step, total debt, interest paid, the effective rate (annual interest divided by debt), and maturities by original tenor are kept.

Identity check (invariant)

At every step the engine checks the stock-flow identity: debt at the end of the month equals debt at the start plus the overall deficit, and nothing else. A deviation of more than one currency unit raises an exception. This guarantees there are no "leaks" in the rollover, as already checked in the POC.

Performance

Tens of thousands of bonds times 120 steps are simple vector operations. Target: one scenario for one country in under 200 milliseconds, so the slider in the interface responds in real time. To

run in the browser as well, the engine is written in Python with NumPy only and loaded in Pyodide.

Complementary industry technologies and infrastructure

- **NumPy (Structure of Arrays):** the pattern game engines and simulation systems use to hold millions of entities. Contribution: updating the whole portfolio in one vector operation instead of a loop over objects.
- **Numba (optional, server side):** a JIT compiler that translates Python functions into machine code. Contribution: if Monte Carlo runs on the server (feature 6) need thousands of runs, the step loop is compiled and runs tens of times faster. Not available in the browser, so it stays optional.
- **Pyodide:** a Python distribution compiled to WebAssembly that runs in the browser, including NumPy. Contribution: the same engine code runs on the server and on the user's machine. It is also the answer to the "taking the service down" threat: computation does not depend on the server.

Data structure

```
class Portfolio:            # Structure of Arrays
    face: np.ndarray        # float64
    coupon: np.ndarray      # float64, % per year
    kind: np.ndarray        # int8: 0 bill, 1 fixed, 2 linker, 3 floater
    issue_m: np.ndarray     # int32, months since epoch
    maturity_m: np.ndarray  # int32
    tenor_m: np.ndarray     # int32, original tenor in months
    maturing_index: dict[int, list[int]]

StepResult {
  month: int
  debt: float
  interest_paid: float
  effective_rate_pct: float
  maturing_by_tenor: dict[int, float]
  issued_by_tenor: dict[int, float]
}
```

Input and output

Input: a snapshot (Bond list) and a Scenario. Output: a list of StepResult for every month, and the final portfolio.

Key libraries and functions

Operation	Library	Usage
Portfolio arrays	NumPy	np.where, np.add.at, masking
Server acceleration	Numba	@njit on the step loop
Browser execution	Pyodide	loadPackage("numpy"), runPythonAsync

Feature 6: Debt Dynamics Engine (r versus g)

What does the feature do?

Translates the rollover engine's output into the questions people care about: what is the debt-to-GDP ratio each year, when does the effective rate overtake growth, and what primary surplus is needed to stop the swelling.

How does it work?

The debt equation

The debt-to-GDP ratio in year t equals the previous year's ratio, times (1 plus the effective rate) divided by (1 plus nominal growth), minus the primary balance, plus stock-flow adjustments (privatisations or loans to public bodies, for example). The effective rate is not assumed; it comes from the rollover engine, year by year. That is the difference between us and an aggregate approximation.

The r versus g crossover

The feature finds the first year in which the effective rate exceeds nominal growth, and marks it. Beyond that point, every year without a primary surplus raises the debt ratio, even with no "new" deficit.

The required primary surplus

For each year: the primary surplus that would stabilise the debt ratio in that year equals (r minus g) divided by (1 plus g), times the debt ratio. The POC showed the formula gives 1.7% for the synthetic portfolio, against the 1.8% The Economist computed for France. The feature also computes the inverse path: "which constant primary balance brings debt back to level X by year Y", using root finding (brentq) on the primary balance, where every attempt reruns the rollover engine.

Uncertainty fan (optional, server side)

Instead of a single scenario, 1,000 paths in which yields and growth are drawn from a joint distribution estimated from history (a simple vector autoregression). The result is a fan chart of the debt ratio, as published by the IMF and the European Commission: percentiles 10, 50 and 90.

Complementary industry technologies and infrastructure

- **SciPy brentq:** a root-finding algorithm combining bisection, secant and interpolation. Contribution: finding the required primary balance in a few dozen runs rather than a blind search.
- **statsmodels (VAR):** a well-known statistics library in which the VAR model is a standard tool for estimating the joint behaviour of several series. Contribution: yield and growth paths that are correlated with each other, as in reality.
- **Sobol sequences (scipy.stats.qmc):** quasi-random sampling that covers the space of possibilities more evenly than ordinary random sampling. Contribution: a stable fan with fewer runs.

Data structure

```
DebtPath {
```

```
  run_id: str
  years: list[int]
  debt_gdp_pct: list[float]
  interest_gdp_pct: list[float]
  effective_rate_pct: list[float]
  growth_pct: list[float]
  r_minus_g_pp: list[float]
  crossover_year: int | None
  stabilizing_pb_pct: list[float]
}

FanChart { run_id; p10, p50, p90: list[float]; n_paths: int; seed: int }
```

Input and output

Input: StepResult from the rollover engine and the scenario's macro assumptions. Output: a DebtPath, and optionally a FanChart.

Key libraries and functions

Operation	Library	Usage
Root finding	SciPy	optimize.brentq
Path model	statsmodels	tsa.api.VAR
Sampling	SciPy	stats.qmc.Sobol

Feature 7: Method Comparison

What does the feature do?

Runs the same scenario with four methods side by side, and shows where they diverge and why. This is the feature that answers the open question from the POC: is bond-level detail actually worth anything. It does not assume the answer; it measures it again on every run.

How does it work?

The four methods

- **Frozen rate:** all debt keeps paying today's effective rate. This is how many back-of-the-envelope calculations work.
- **Instant jump:** all debt moves at once to the market rate. This is The Economist's thought experiment.
- **IMF-style approximation:** an effective rate updated each year according to the share of debt being renewed, with the share derived from today's maturity profile. An advanced version separates short from long debt, as in the IMF DSA template.
- **Bond by bond:** the full rollover engine (feature 5).

Measuring the gap

For each year: the gap in interest payments (as % of GDP) between each method and the full engine, and the gap in the debt ratio. The feature marks the year with the largest gap and

decomposes it by source: how much comes from rolling new short bonds (which the approximation does not see), how much from linkers, and how much from monthly timing.

What we already checked

Before the spec we ran an IMF-style approximation (the simple version, without the short/long split) on the POC's synthetic portfolio. The gap against the full engine reached 0.53 points of GDP in the "yields stay" scenario, 0.8 in the "up" scenario, and 0.29 in the "down" scenario. The debt ratio in 2036: 157% against 154.5% in the base scenario. The advanced version of the approximation may narrow the gap, and that is exactly why the feature runs it and shows the result openly, even if it shows the gap is small.

Complementary industry technologies and infrastructure

- **IMF MAC SRDSF (as a methodological source):** the IMF's debt-sustainability framework for market-access countries, whose template is public. Contribution: our approximation follows its documented methodology, so the comparison is fair rather than a straw man.
- **Property-based testing with Hypothesis:** a testing library that generates thousands of random inputs and checks that a property always holds. Contribution: a test that in a portfolio where every bond has the same tenor, three of the four methods must converge on each other. If they do not, there is a bug.

Data structure

```
MethodRun { run_id; method: Literal["frozen", "jump", "imf_style", "imf_split", "bond"];
path: DebtPath }

MethodDelta {
  run_id: str
  method: str
  year: int
  interest_gap_pp: float      # method minus bond-level, % GDP
  debt_gap_pp: float
  gap_from_short_rollover: float
  gap_from_linkers: float
  gap_from_timing: float
}
```

Input and output

Input: a snapshot and a scenario. Output: five DebtPath, a MethodDelta table, and the year of the largest gap.

Key libraries and functions

Operation	Library	Usage
Parallel runs	concurrent.futures	ProcessPoolExecutor on the server
Property tests	Hypothesis	@given(strategies)

Feature 8: Maturity Wall View

What does the feature do?

The interface where the user meets all of this. The central view is a "wall": each bond is a brick, bricks are arranged by maturity year, and each column's height is how much debt matures that year. The yield slider colours the bricks by how much more it will cost to replace them. Below the wall: the debt-ratio and interest paths, and the method comparison.

How does it work?

Colouring the bricks

For each bond a "coupon jump" is computed: the yield at which it will be replaced under the scenario, minus its current coupon. Colour follows a diverging scale around zero: cool tones for bonds that will be replaced more cheaply, warm tones for those replaced at higher cost. The scale is colour-blind safe, and every piece of information carried by colour also appears as text (tooltip and table).

Interaction

- **Yield slider:** a change reruns the rollover engine (in the browser, in a Web Worker so the interface does not freeze), with a 150-millisecond debounce.
- **Hover or focus on a brick:** a popup with the identifier, coupon, maturity date and expected replacement.
- **Compare mode:** two countries side by side on the same axis.
- **Accessibility:** every brick is keyboard accessible, Esc closes a popup, and next to the wall is a full text table of the same information.

Assumptions stay attached to the result

In answer to the "misuse of the result" threat: every chart carries a subtitle with the scenario and assumptions ("yield +1 point, growth 2%, primary deficit 2.9%"). A screenshot of the chart always includes them, not as a footnote. There is no score and no rating for a country.

Public API

For journalists and developers: FastAPI with endpoints such as GET /v1/countries/{code}/snapshots/latest, POST /v1/runs (takes a scenario and returns a run_id), GET /v1/runs/{run_id}. Automatic OpenAPI documentation. Rate limiting per IP address, and results cached by run_id, since the same input always gives the same result.

Complementary industry technologies and infrastructure

- **D3.js:** the world's most widely used library for custom visualisations, used by the newsrooms of The New York Times and The Economist itself. Contribution: a custom brick wall that an ordinary charting library cannot build.
- **Web Workers:** a browser mechanism for running code in the background, on a thread separate from the interface. Contribution: the rollover engine on Pyodide runs without blocking scrolling or the slider.

- **FastAPI with slowapi:** a modern Python API framework that generates OpenAPI documentation automatically, with slowapi adding rate limiting. Contribution: an answer to the "taking the service down" threat on the API side.

Data structure

```
WallBrick { instrument_id; maturity_year; face; coupon_pct; replacement_yield_pct; jump_pp }
WallColumn { year; total_face; bricks: list[WallBrick] }
ViewState { country; scenario_id; compare_country?: str; selected_brick?: str }
```

Input and output

Input: a snapshot, a Scenario, and DebtPath and MethodDelta from features 6 and 7. Output: an interactive view, an accessible table, and API responses.

Key libraries and functions

Operation	Library	Usage
Visualisation	D3.js	d3.scaleDiverging, d3.stack
Background execution	Web Workers	postMessage / onmessage
API	FastAPI, slowapi	APIRouter, Limiter

Feature 9: Reproducible Permalinks & Signed Results

What does the feature do?

Every result has a permanent link that reproduces it exactly, and a digital signature proving it came from the system. This answers two threats from the threat model: "repudiating the source" (no way to know where a number came from) and "spoofing" (a fake screenshot carrying the tool's logo).

How does it work?

A deterministic run identifier

run_id is the SHA-256 of three things together: snapshot_id, scenario_id and the engine version. Since all three are deterministic, the same run_id always describes exactly the same computation. The link is simply /run/{run_id}. Whoever opens it gets exactly the same result, even a year later, because the snapshot never changes (feature 3).

Signing the result

Every result is packaged as a ResultBundle: the run_id, the headline numbers and the assumptions. The bundle is serialised to canonical JSON and signed with an Ed25519 signature using the system's private key. The public key is published on the site. Anyone can verify that a given number really came from the system under those assumptions. An image downloaded from the interface carries a small QR code in the corner linking to the run, so a fake screenshot can be checked against the source.

Key management

The private key does not sit in a file on the server: it is held in a cloud key-management service that performs the signature inside itself and returns only the result. The key rotates once a year, and all previous keys remain published so old signatures can be verified.

Complementary industry technologies and infrastructure

- **PyNaCl (Ed25519):** a Python wrapper around libsodium. Ed25519 is fast, its signatures are short (64 bytes), and it is the standard in modern tools. Contribution: signing and verification in milliseconds.
- **Cloud KMS (AWS KMS or Google Cloud KMS, for example):** a service in which the private key never leaves the service. Contribution: even a complete server compromise does not allow the key to be stolen.
- **Sigstore (as inspiration):** an open project for signing software and artefacts, with a public transparency log. We adopt the idea of a log: every signed result is recorded in an append-only log, so even the system itself cannot "sign retroactively" and make something disappear.

Data structure

```
ResultBundle {
  run_id: str
  snapshot_id: str
  scenario_id: str
  engine_version: str
  headline: { debt_2036_pct, interest_2031_pct, crossover_year }
  assumptions_text: str
}
SignedResult { bundle: ResultBundle; key_id: str; signature: bytes[64]; signed_at:
datetime }
```

Input and output

Input: results from features 5 to 7. Output: a permanent link, a signed bundle, and an entry in the transparency log.

Key libraries and functions

Operation	Library / service	Usage
Sign and verify	PyNaCl	SigningKey.sign(), VerifyKey.verify()
Key custody	Cloud KMS	Sign API, the key never leaves
QR	qrcode	make(url)

Feature 10: Scheduled Refresh & Change Detection

What does the feature do?

Runs the whole chain (fetch, normalise, check, store) automatically after every debt-office publication, and alerts when something has changed unusually: a jump in the maturity wall, a large new issue, or a load that failed the completeness check.

How does it work?

Scheduling

Each country gets a schedule matching its source's publication calendar (the American MSPD and the European debt offices' files are published monthly, each on its own calendar). The refresh runs as a workflow in which each stage is a separate task, with retries and dependencies: no storing without checking, and no checking without downloading.

What counts as an unusual change

After every approved load, the feature compares the key metrics with the previous load: the size of the wall over the next three years, the average coupon of debt maturing in the next two years, and the effective rate. A change of more than 3 standard deviations from the historical monthly change creates an alert. A REJECTED load always creates an alert, because it means a source changed and the adapter may need updating.

A public changelog

In answer to the "changing the defaults" threat: every change to the default scenario settings, the issuance mix or the engine version is recorded in a public changelog with a date, a description and an explanation. The defaults themselves live in code, so every change goes through code review.

Complementary industry technologies and infrastructure

- **Prefect:** a modern Python workflow orchestrator in which each stage is a task with retries, dependencies and run records. Contribution: a refresh that fails midway resumes from the failed stage, and every run is documented. A simpler alternative: GitHub Actions with cron.
- **Apprise:** one library that sends notifications to dozens of channels (email, Slack, Telegram). Contribution: the alert channel is configuration, not code.
- **Keep a Changelog:** a well-known convention for documenting changes in open projects. Contribution: a changelog a human can read.

Data structure

```
RefreshRun { refresh_id; country; started_at; finished_at; status: Literal["OK",
"REJECTED", "FAILED"]; snapshot_id?: str }
ChangeAlert { alert_id; country; metric; old_value; new_value; z_score; created_at }
ChangelogEntry { entry_id; date; area: Literal["defaults", "mix", "engine", "adapter"];
description; commit_sha }
```

Input and output

Input: a schedule per country. Output: a new snapshot or a failure alert, unusual-change alerts, and changelog entries.

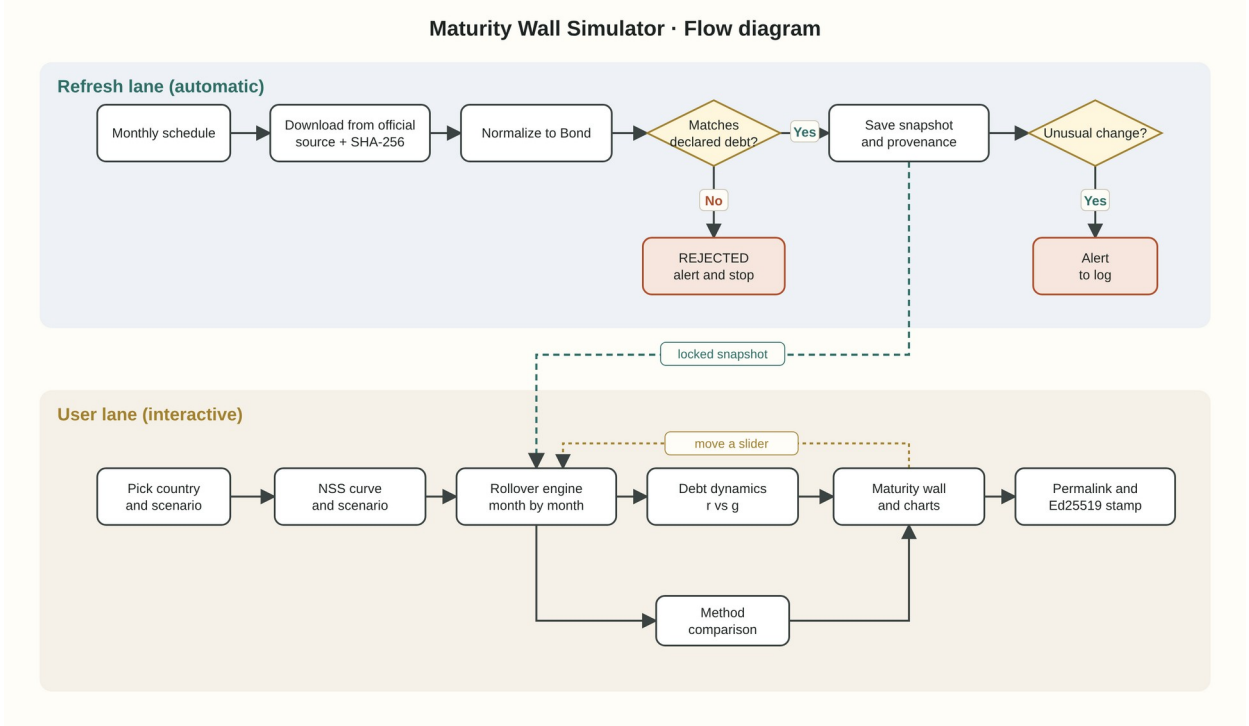
Key libraries and functions

Operation	Library	Usage
Orchestration	Prefect	@flow, @task(retries=3)
Notifications	Apprise	Apprise().notify()
Simple scheduling	GitHub Actions	on: schedule: cron

Technology table

Feature	Technologies and languages	External resources and services
1. Source adapters	Python, httpx, tenacity, openpyxl, Pydantic	Fiscal Data API, ECB Data Portal, AFT and Italian Treasury files
2. Normalisation and checks	Python, decimal, Pandera, Polars	None
3. Store and provenance	DuckDB, Parquet, hashlib	File storage (local or S3)
4. Curve and scenarios	NumPy, SciPy, QuantLib	ECB parameters, IMF and Commission forecasts
5. Rollover engine	NumPy, Numba, Pyodide	None (also runs in the browser)
6. Debt dynamics	SciPy, statsmodels	None
7. Method comparison	Python, Hypothesis	IMF MAC SRDSF methodology
8. Maturity wall	D3.js, Web Workers, FastAPI, slowapi	CDN for static files
9. Link and signature	PyNaCl, qrcode	Cloud KMS
10. Scheduled refresh	Prefect, Apprise	Small server or GitHub Actions

Flow diagram



The diagram shows the system's two lanes. The refresh lane (top) runs automatically: download, normalise, completeness check, then a decision. A failed load raises an alert and stops, with no path into the store. A load that passes is saved as a snapshot, and unusual changes are checked from there. The user lane (bottom) starts with picking a country and scenario, passes through the rollover engine and the debt engine, and in parallel the method comparison, and ends with the wall, the permalink and the signature. The loop back from the slider to the rollover engine is the interactive cycle: every move reruns the engine, in the browser.

Database structure

The database is implemented with DuckDB, a columnar analytical database running in-process, with raw files in fingerprint-keyed storage. All tables are append-only: no UPDATE and no DELETE.

Table fetches: every downloaded raw file

- **fetch_id:** VARCHAR, primary key (UUID).
- **source_id:** VARCHAR, required (for example "US_MSPD_T3").
- **url:** VARCHAR, required.
- **fetched_at:** TIMESTAMP, required.
- **raw_sha256:** VARCHAR(64), required. The file path is derived from it.
- **http_status:** SMALLINT.

Table snapshots: every approved load

- **snapshot_id:** VARCHAR(64), primary key (SHA-256 of the canonical list).
- **country:** VARCHAR(2), required.
- **as_of:** DATE, required.
- **created_at:** TIMESTAMP, required.
- **bond_count:** INTEGER, required.
- **total_face:** DECIMAL(20,2), required.
- **reconciliation_gap_pct:** DOUBLE, required, between 0 and 0.5.
- **adapter_version:** VARCHAR(40), required (git commit).
- **Constraint:** unique on (country, as_of, snapshot_id).

Table snapshot_sources: many-to-many between snapshots and files

- **snapshot_id:** foreign key to snapshots.
- **fetch_id:** foreign key to fetches.
- Composite primary key on both.

Table bonds: the bonds of each snapshot

- **snapshot_id:** foreign key to snapshots, required.
- **instrument_id:** VARCHAR(12), required (CUSIP or ISIN).
- **kind:** ENUM('bill','fixed','linker','floater'), required.
- **coupon_pct:** DECIMAL(6,4), may be NULL for bills.
- **issue_date:** DATE, required.
- **maturity_date:** DATE, required, with a CHECK that it is after issue_date.
- **face:** DECIMAL(20,2), required, with a CHECK that it is positive.
- **orig_tenor_m:** SMALLINT, required.
- **Composite primary key:** (snapshot_id, instrument_id).

Table validation_issues: check results

- **issue_id:** VARCHAR, primary key.
- **refresh_id:** foreign key to refresh_runs.
- **rule_id:** VARCHAR, required.
- **severity:** ENUM('WARN','BLOCK'), required.
- **instrument_id:** VARCHAR, may be NULL (a check on the whole load).
- **detail:** VARCHAR, required.

Table curves: yield curves

- **curve_id:** VARCHAR(64), primary key.
- **base:** ENUM('ECB_AAA','ECB_ALL','UST_PAR','FITTED'), required.
- **as_of:** DATE, required.
- **params:** STRUCT(beta0, beta1, beta2, beta3, tau1, tau2 DOUBLE), required.
- **spread_params:** the same structure, may be NULL.
- **fit_rmse_bp:** DOUBLE.

Table scenarios: scenario definitions

- **scenario_id:** VARCHAR(64), primary key (SHA-256 of the canonical definition).
- **name:** VARCHAR, required.
- **curve_id:** foreign key to curves.
- **definition:** JSON, required (shocks, growth, primary balance, mix).
- **is_default:** BOOLEAN, default false.

Table runs: every run

- **run_id:** VARCHAR(64), primary key (SHA-256 of snapshot, scenario and version).
- **snapshot_id:** foreign key to snapshots.
- **scenario_id:** foreign key to scenarios.
- **engine_version:** VARCHAR, required.
- **created_at:** TIMESTAMP.

Table run_paths: the annual path of every run and method

- **run_id:** foreign key to runs.
- **method:** ENUM('frozen','jump','imf_style','imf_split','bond'), required.
- **year:** SMALLINT, required.
- **debt_gdp_pct, interest_gdp_pct, effective_rate_pct, r_minus_g_pp:** DOUBLE.
- **Composite primary key:** (run_id, method, year).

Table signed_results: the signature log

- **run_id:** foreign key to runs, primary key.
- **bundle:** JSON, required.
- **key_id:** VARCHAR, required.

- **signature:** BLOB (64 bytes), required.
- **signed_at:** TIMESTAMP, required.
- **prev_hash:** VARCHAR(64), required. The fingerprint of the previous record, so the log is a chain from which no record can be dropped without detection.

Tables refresh_runs, change_alerts and changelog

- **refresh_runs:** refresh_id (primary key), country, started_at, finished_at, status (ENUM: OK, REJECTED, FAILED), snapshot_id (may be NULL).
- **change_alerts:** alert_id (primary key), country, metric, old_value, new_value, z_score, created_at.
- **changelog:** entry_id (primary key), date, area (ENUM: defaults, mix, engine, adapter), description, commit_sha.

What the spec closed, and what remains open

- **Closed:** full provenance from every number to the official file (features 1 to 3, 9), an answer to all six threats in the threat model, and the POC's question turned into a feature that measures itself on every run (feature 7).
- **Open:** our check shows a real gap against the simple version of an IMF-style approximation. Does the advanced version, which separates short and long debt, close most of the gap? If so, the system's advantage shrinks to accessibility and transparency alone.
- **Open:** the system knows exactly how much each country will pay and when. It does not know who holds those bonds, and who will be hurt when their prices move.